

Dynamics and Learning of Interacting Particle Systems

Nipuni de Silva, Uresha Dias

Advisor: Prof. James Greene



Clarkson University – RAPS-2024
Department of Mathematics

Definition(Self Organization/ Collective Behavior)

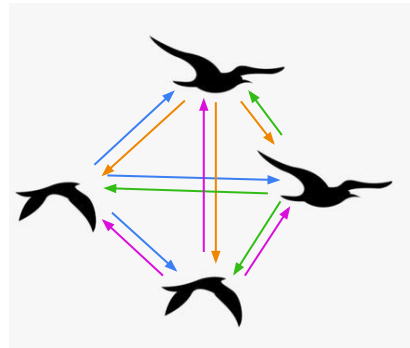
It is a process where some global order emerges from an initially disordered system via local interactions.



School of fish



Flocking birds



Pairwise interaction
between agents



Applications:

- Particle Swarm Optimization
- Consensus Analysis and Social Behavior
- Animal Migration and Conservation
- Unmanned aerial vehicles(UAV)
- Self driving cars

Examples of Self-Organization/Collective Behavior in Different Fields:

- **Physics:** Spontaneous magnetization, crystal growth, superconductivity
- **Chemistry:** Molecular self-assembly, self-assembled monolayers
- **Biology:** Formation of lipid bilayer membranes, social behavior in insects, flocking of birds
- **Human Society:** Market economy, traffic flow

Modeling Collective Motion/Behavior

Consider a system of N agents, each of which is assigned $x_i \in \mathbb{R}^d$,

$$\frac{dx_i}{dt} = \frac{1}{N} \sum_{j=1}^N \phi(|x_j - x_i|)(x_j - x_i), \quad i = 1, \dots, N$$

N := Number of agents

ϕ := Interaction Kernel

$\dot{x}_i(t)$:= Velocity of i^{th} agent at time t

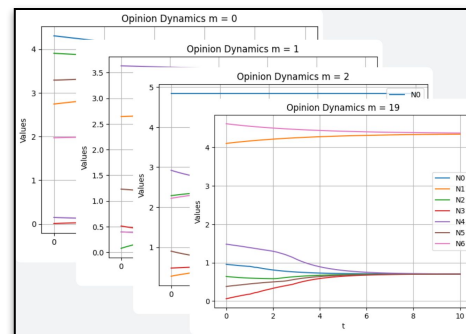
$\|x_j(t) - x_i(t)\|$:= Distance between i^{th} and j^{th} agent at time t

ϕ defines the dynamics of the system

What we are interested in,

- Approaches to learn the interaction kernel ϕ using observed trajectory data without making any assumptions about their analytical form.
- The advantage of this interaction kernel-based approach is that it incorporates the physics of the model, which other approaches may not consider, potentially limiting their effectiveness

We are looking at an inverse problem.
Given $\{x_i(t), \dot{x}_i(t)\}_{j=1}^N$ for $t \in [0, T]$,
can ϕ be learned?

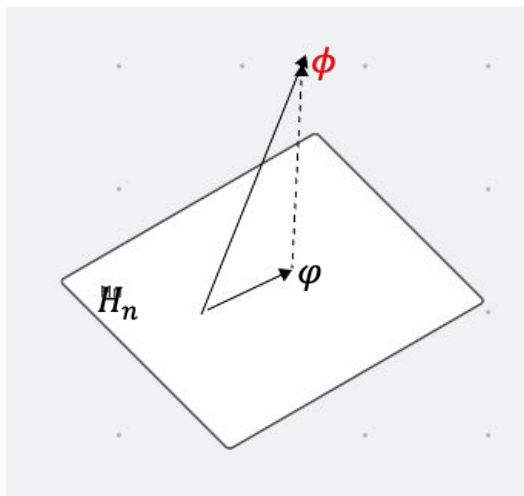


What Methods Can be Used to Learn Interaction Kernel in 1st Order Deterministic Systems.

The algorithms target to find a coefficient set $\mathbf{a} = [a_1, a_2, \dots, a_n]$ such that

$$\phi \approx \varphi = a_1\varphi_1 + a_2\varphi_2 + \dots + a_n\varphi_n$$

in a defined hypothesis space H_n .



Nonparametric Learning Method

- Learn the interaction kernel ϕ of the N agent system using M independent trajectories.
- Obtain the estimator φ by minimizing the error functional over all functions φ^* in the defined hypothesis space H_n . Here $r_{ij} = (x_j - x_i)$

$$\varepsilon_{L,M}(\varphi^*) = \frac{1}{LMN} \sum_{l,m,i} \left\| \dot{x}_i^m(t_l) - \frac{1}{N} \sum_{j=1}^N \varphi^*(|r_{ij}^m(t_l)|) (r_{ij}^m(t_l)) \right\|^2$$

$\mathcal{E} :=$ Discrete error functional

$L :=$ Number of time steps

$M :=$ Number of trajectories

$\dot{x}_i(t) :=$ Velocity of i^{th} agent at time t

- Rewrite error functional

$$\varepsilon_{L,M}(\varphi^*) = \varepsilon_{L,M}(\mathbf{a}) = \frac{1}{LMN} \sum_{m=1}^M \|d^m - \Psi_L^m \mathbf{a}\|_{\mathbb{R}^{LNd}}^2$$

- This leads to a least-square problem for $\mathbf{a}$

$$\left(\frac{1}{M} \sum_{m=1}^M A_L^m \right) \mathbf{a} = \left(\frac{1}{M} \sum_{m=1}^M b_L^m \right)$$

where the trajectory-wise regression matrices are $A_L^m = \frac{1}{LN} (\Psi_L^m)^T \Psi_L^m$ and $b_L^m = \frac{1}{LN} (\Psi_L^m)^T d^m$

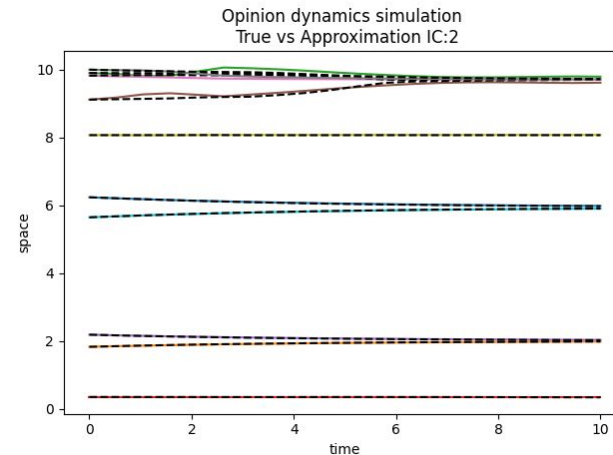
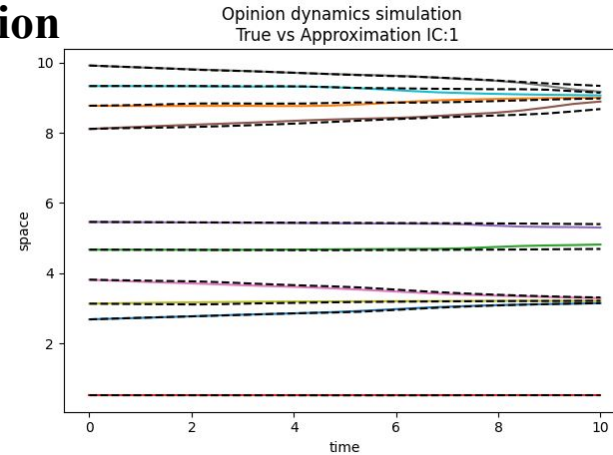
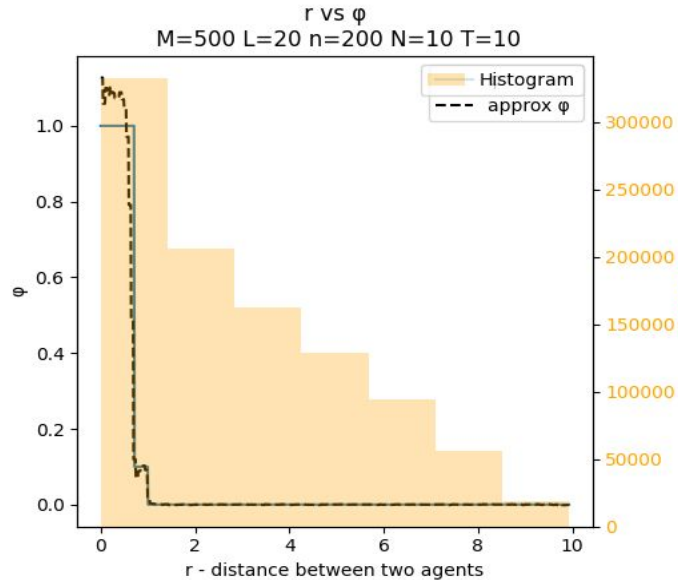
Sparse Representation Learning Method

- Structure d^m vector and Ψ_L^m matrix.
- Apply optimization step of sparse representation algorithm.

$$\mathbf{a} = \underset{\mathbf{a}}{\operatorname{argmin}} \|\Psi_L^m \mathbf{a} - d^m\|_2 + \lambda \|\mathbf{a}\|_1$$

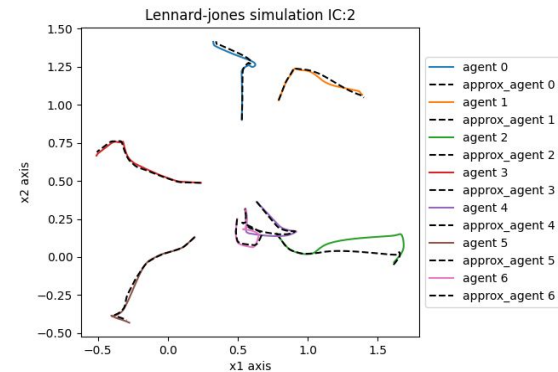
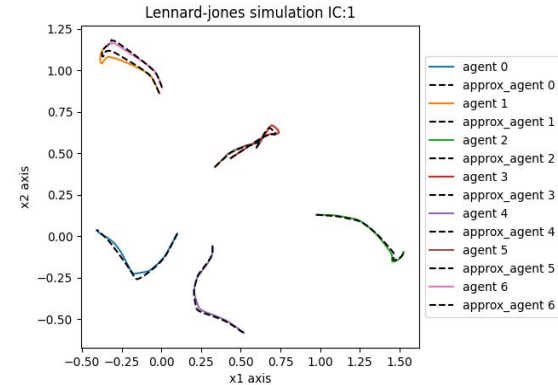
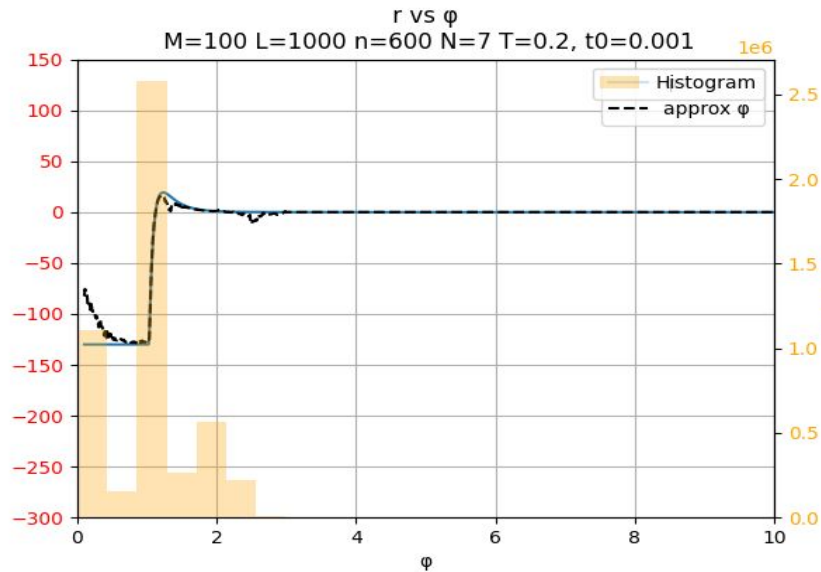
Nonparametric Learning Method – 1D Simulation

$$\phi(r) = \begin{cases} 1, & 0 \leq r \leq 1/\sqrt{2} \\ 0.1, & \frac{1}{\sqrt{2}} \leq r \leq 1 \\ 0, & 1 \leq r \end{cases}$$



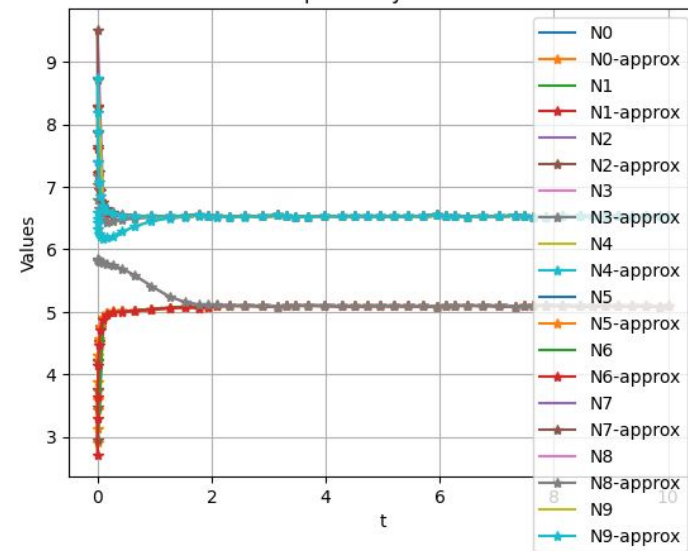
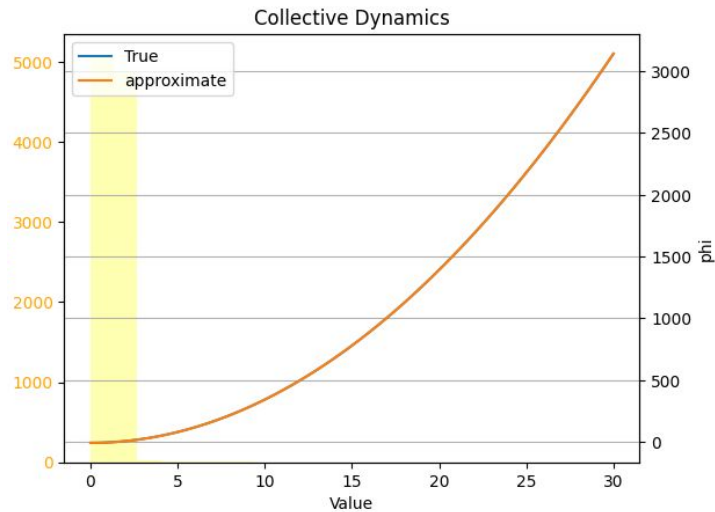
Nonparametric Learning Method – 2D Simulation

$$\Phi(r) = 4\epsilon \left[\left(\frac{\sigma}{r} \right)^{12} - \left(\frac{\sigma}{r} \right)^6 \right] \text{ and } \phi(r) = \frac{\Phi'(r)}{r}, \epsilon = 10, \sigma = 1$$



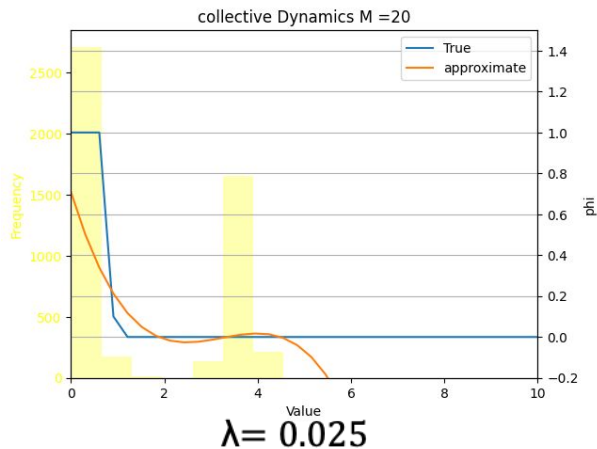
Sparse Representation learning method- Simulation

$$\phi(r) = 3.5r^2 - 7.1$$



Which algorithm to use?

- The sparse representation learning method comes with one assumption: the algorithm works well with functions that exhibit sparsity within the given hypothesis space.
- Therefore, this method won't be accurate for non-sparse function approximations.



- The sparse representation learning method is computationally fast and requires fewer observations to output the approximation.

Conclusion

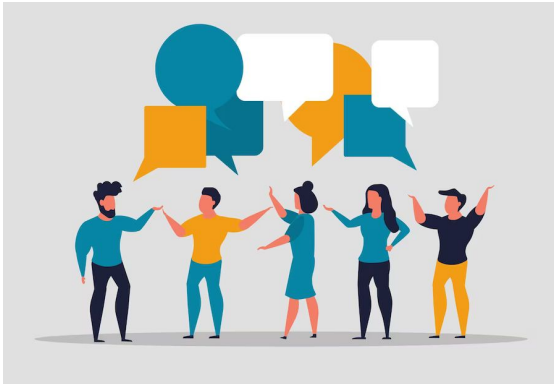
- Both algorithm build to works with observation data to learn interaction kernel and choice of hypothesis space affect the output.
- Sparse representation method is efficient and best with kernels with sparsity.
- Nonparametric learning method can handle almost any interaction kernel function regardless of any assumption (at least if smooth enough)
- Also , this method is computationally expensive than sparse representation method .

Future work

- Quantifying the tradeoff between sparsity accuracy and the amount of data.
- Investigating the method on kernels that are non smooth (Singular) .
- Comparison algorithms with other classical machine learning methods. (NODE, NN)

Reference

1. Ashby, W. R. (1947). Principles of the Self-Organizing Dynamic System. *The Journal of General Psychology*, 37(2), 125–128.
<https://doi.org/10.1080/00221309.1947.9918144>
2. Feng, J., Maggioni, M., Martin, P., & Zhong, M. (2022). Learning interaction variables and kernels from observations of agent-based systems. *IFAC-PapersOnLine*, 55(30), 162-167.
3. Lu, F., Zhong, M., Tang, S., & Maggioni, M. (2019). Nonparametric inference of interaction laws in systems of agents from trajectory data. *Proceedings of the National Academy of Sciences*, 116(29), 14424-14433.
4. Brunton, Steven L., Joshua L. Proctor, and J. Nathan Kutz. "Discovering governing equations from data by sparse identification of nonlinear dynamical systems." *Proceedings of the national academy of sciences* 113.15 (2016): 3932-3937.



Thank You!